

HyperRAG: Using Hyperlinks Between Web Pages to Help RAG Answer Multi-Hop Questions

Ravi Prajapati

Faculty of Computing

Islington College (London Metropolitan University)

Kathmandu, Nepal

np01ms7a250053@islingtoncollege.edu.np

Abstract—Most Retrieval-Augmented Generation (RAG) systems pick web pages one by one and score each page on its own. The real web is not like that. Wikipedia and most wiki or news sites are full of hyperlinks placed by editors, and a question whose answer is split across two pages will often break a normal retriever. This paper proposes HyperRAG, a small extension on top of standard dense retrieval. After the FAISS top- K search returns the first set of pages, the system takes one short walk along the HTML hyperlink graph (both outgoing and incoming edges) and pulls in the bridge pages that pure embedding similarity tends to miss. The method needs no extra training, no LLM-built knowledge graph and no new annotation, because the edges already sit inside the HTML as `<a href>` tags. We test on 7,926 HotpotQA bridge questions over a Wikipedia corpus of 24,473 pages. Six chunk sizes are tried (512, 1024, 2048, 4000, 8000 characters and full-page) and Qwen-3.5-9B is used as the answering LLM, giving 142,668 evaluated rows. HyperRAG leads on Supporting Recall at every chunk size (0.683 to 0.754, against 0.561 to 0.584 for the two baselines), it leads on Retrieval Exact Match at every chunk size (up to 0.755 at full-page) and it leads on Generation Exact Match and Generation F1 at every chunk size up to 8K characters (up to 0.445 and 0.442 respectively). The graph step costs one extra similarity search per query, and across the whole sweep it never makes the system worse.

Index Terms—Retrieval-Augmented Generation, Multi-Hop QA, Hyperlink Graph, Chunking, HotpotQA, Dense Retrieval, NetworkX, FAISS, Qwen

I. BACKGROUND OF THE STUDY

Large Language Models like GPT, Llama and Claude can write fluent answers, but they also make things up sometimes. The standard fix used in industry today is Retrieval-Augmented Generation, or RAG [1]. Before the model writes an answer, a retriever pulls a few relevant pages out of a corpus and the LLM is told to use those pages as the source. RAG works fine when one page is enough. The trouble shows up when the answer needs two pages.

A well-known example from HotpotQA [2] is the question “*The Oberoi family is part of a hotel company that has its head office in what city?*”. The right answer is *Delhi*, but to get there the system needs page A about the Oberoi family (which says they own The Oberoi Group) and page B about The Oberoi Group itself (which says the head office is in Delhi). A dense retriever scores each page on its own. Page A matches the

wording of the question. Page B does not, because the question never mentions “The Oberoi Group” by name. So page B never enters the context, and the LLM has nothing to work with.

Standard RAG misses bridge pages because it ignores one simple thing: web pages link to each other. Wikipedia is basically a graph. Every `<a href>` tag is a directed edge that an editor placed there because two articles are related. HyperRAG just uses these existing links as a free retrieval signal that nobody has paid to build.

The aim of this study is simple. Once dense retrieval has picked its top- K pages, take one step along the hyperlink graph, pull in the neighbours, and check if this small extra step really improves multi-hop retrieval and the final LLM answer on HotpotQA. The contributions of this paper are:

- A full working pipeline (data fetching, corpus building, chunking, FAISS index, hyperlink graph, retrieval, LLM generation, evaluation) released on GitHub so that the experiment can be reproduced.
- A 24,473-page Wikipedia distractor corpus and an evaluation set of 7,926 HotpotQA bridge questions, used in the same way for all three systems.
- A six-point chunk-size sweep (512, 1024, 2048, 4000, 8000 characters and full-page) crossed with three retrieval systems (Simple RAG, HtmlRAG, HyperRAG), giving 142,668 evaluated rows once retrieval and generation metrics are combined.
- End-to-end generation results using Qwen-3.5-9B as the reader, reporting Generation Exact Match and Generation F1 alongside Retrieval EM, Retrieval F1 and Supporting Recall.
- A short discussion of three things the data shows clearly: the gap between retrieval EM and generation EM, the saturation of generation quality above 2–4K characters, and why HtmlRAG’s structural markup actually hurts more than it helps in this setup.

The rest of the paper is laid out like this. Section II reviews the closest related work. Section III explains the pipeline step by step. Section IV reports the results across the chunk-size sweep. Section V closes with what worked, what did not, and what comes next.

II. RELATED WORK

The work in this area falls into three groups: plain RAG and dense retrieval, HTML-aware retrieval, and graph-augmented retrieval. The last subsection explains how HyperRAG is different from each of them.

A. Plain RAG and Dense Retrieval

Lewis et al. [1] introduced the RAG framework, where the generator is conditioned on documents fetched from a non-parametric memory. The standard retriever is Dense Passage Retrieval [3], which embeds the query and the passages with two BERT towers and uses inner product over a FAISS [4] index. The score is simply

$$s(q, d) = \phi_Q(q)^\top \phi_D(d), \quad (1)$$

with the vectors L2-normalised so that inner product equals cosine similarity. Bi-encoder fine-tuning [5], iterative retrieval and query rewriting have all been added on top later, but the basic shape has not changed: every passage is scored on its own.

B. HTML-Aware Retrieval

A web corpus is not just plain text. The HTML carries headings, tables and lists, and stripping all of that throws away information the LLM could probably use. HtmlRAG [6] keeps cleaned HTML as the retrieval unit instead of stripped text, and reports gains on open-domain QA. The B2 baseline used in this paper follows that recipe: keep the `<h1>`, `<h2>`, `<p>`, `<li>` and `<table>` tags, and remove `<script>`, `<style>`, `<nav>` and `<sup>` (the footnote markers).

C. Graph-Augmented Retrieval

Several recent systems add a graph layer on top of retrieval. Microsoft GraphRAG [7] runs an LLM over the corpus to extract entities and relations, builds a knowledge graph, and answers global questions by summarising the graph communities. HippoRAG [8] uses Personalised PageRank over an entity graph and reports big gains on multi-hop QA. HopRAG [9] trains an LLM to add edges as it answers. RAPTOR [10] clusters chunks into a tree of summaries.

The thing all four share is that the graph has to be *built*. GraphRAG and HopRAG need LLM passes over every document. HippoRAG and RAPTOR need entity extraction or recursive summarisation. The graph in those systems is a derived artifact, not a property of the source corpus itself.

D. Multi-Hop QA Benchmarks

HotpotQA [2] is the standard benchmark used for multi-hop QA. Every question is annotated with the supporting Wikipedia pages, so it is possible to score whether the retriever actually pulled in the right evidence. Another line of work tries to handle multi-hop questions by interleaving retrieval with chain-of-thought reasoning, such as IRCOT [15], instead of changing the retriever itself. The closest paper in spirit to HyperRAG is Asai et al. [11], who back in 2020 trained a recurrent retriever to walk the Wikipedia hyperlink graph

for multi-hop QA. Their work predates the modern dense-retrieval-plus-LLM stack. They paired the graph with TF-IDF or DPR initial retrieval and a custom RNN reader. HyperRAG borrows the same hyperlink idea but plugs it into a normal sentence-transformer + FAISS + LLM RAG pipeline, so the graph signal can be measured cleanly on its own.

E. The Gap

Across the three groups above, no system uses the hyperlink graph that already exists inside HTML as a retrieval signal in a modern RAG pipeline. GraphRAG and similar systems build graphs from extracted entities. HtmlRAG keeps HTML structure but only *within* a page. Plain DPR ignores both. The hyperlink graph itself is free: editors of Wikipedia have already drawn the edges. HyperRAG is the small extension that uses what is already sitting there.

III. METHODOLOGY

The pipeline is shown in Fig. 1. Each box maps to one function in the codebase. The full source is available in the project repository [18].

A. Phase 1 – Dataset and Corpus

HotpotQA is loaded through the HuggingFace `datasets` library in its `fullwiki` configuration. Only *bridge*-type questions are kept, because those are the ones where the answer needs two pages and the graph step has any chance of helping. The 24,473-page corpus is sampled from the HotpotQA distractor pool together with the unique supporting titles of the chosen questions. Every page is stored with four fields: `title`, `text` (plain), `html` (cleaned but with structure), and `links` (the `/wiki/` targets found inside the HTML).

The evaluation set contains 7,926 bridge questions. Each question has a gold answer and two gold supporting Wikipedia titles. The same 7,926 questions are used for every system and every chunk size, so the only thing that changes between rows is the system and the chunk size. This keeps the comparison fair.

B. Phase 2 – Hyperlink Graph

The graph is a NetworkX [13] directed graph. Every page is a node. For each page u , every link to a page v that also lives in the corpus becomes a directed edge $u \rightarrow v$. Links to pages that are not in the corpus are dropped. Namespace links (`File:`, `Category:`, `Talk:`, `Help:`) are filtered out because they are not real article-to-article connections. The result is a sparse but topical graph: pages are connected to other pages that an editor thought were relevant. A small region of the graph is shown in Fig. 2.

C. Phase 3 – Embeddings and FAISS

Every retrieval unit (a chunk in the chunked study, a whole page in the full-page setting) is embedded with `sentence-transformers/all-MiniLM-L6-v2` [12], which gives a 384-dimensional vector. The vectors are L2-normalised and stored in a FAISS `IndexFlatIP`, so that

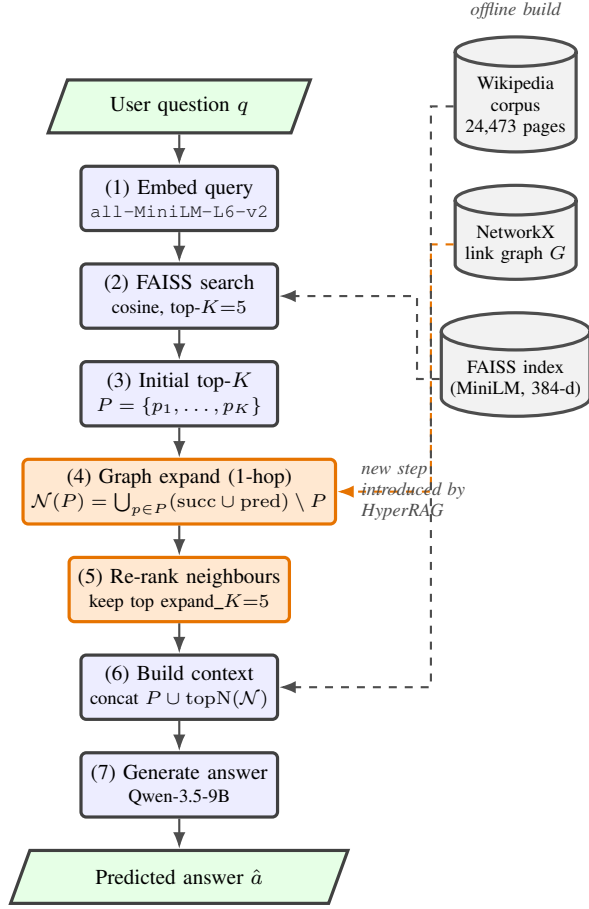


Fig. 1. The HyperRAG pipeline. Steps (1)–(3) and (6)–(7) are shared with a normal RAG system. Steps (4) and (5), drawn in orange, are the only new operations HyperRAG adds: a one-hop walk over the NetworkX hyperlink graph, followed by a re-rank of the new neighbours against the same query embedding. The dashed lines on the right show the offline artifacts (the Wikipedia corpus, the link graph G and the FAISS index) that the runtime pipeline uses.

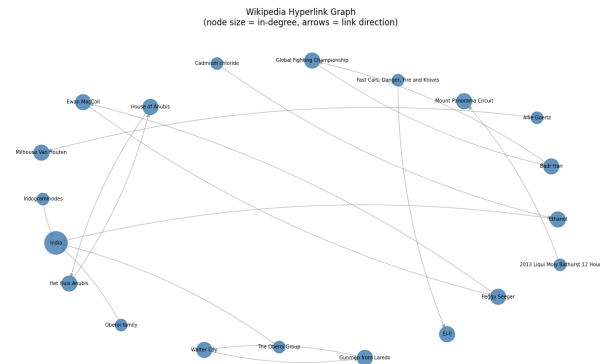


Fig. 2. A sub-region of the Wikipedia hyperlink graph used for retrieval. Each node is a page, and each directed edge is a native HTML `<a href>` link.

inner product equals cosine similarity (Eq. (1)). For every query the same model embeds the question, and FAISS returns the top- $K = 5$ items.

D. Phase 4 – The Three Retrieval Systems

The three systems all share the FAISS step. They only differ in what they do after it.

- **B1 – Simple RAG:** return the plain text of the top- K items.
- **B2 – HtmlRAG:** return the cleaned HTML of the top- K items. Same retrieval as Simple RAG, just a different context format.
- **HyperRAG (this work):** after FAISS picks the top- K items, walk one hop along the page-level graph in both directions (out-edges *and* in-edges). Take the new neighbours that are not already in the top- K , embed them, score them against the query, and append the top `expand_K = 5` of them to the context.

The set of new candidates HyperRAG considers is

$$\mathcal{N}(P) = \bigcup_{p \in P} (\text{succ}_G(p) \cup \text{pred}_G(p)) \setminus P, \quad (2)$$

where P is the set of K items FAISS returned. The original K items are always kept, so HyperRAG is at least as good as Simple RAG in coverage by construction. It can only add pages, it cannot remove them. Fig. 3 traces a single multi-hop question through the pipeline and shows which pages came from FAISS and which were pulled in by the graph step.

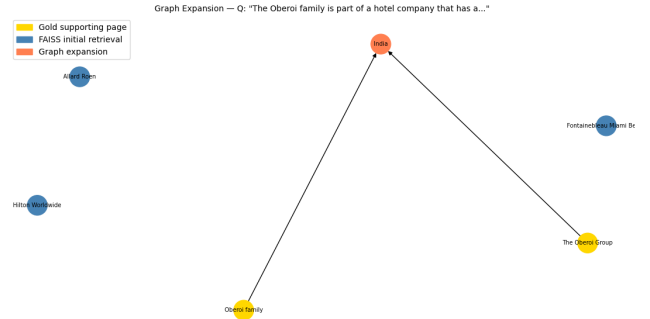


Fig. 3. One-hop expansion in action. Blue = FAISS top- K . Orange = pulled in by the graph step. Yellow = gold supporting page.

E. Phase 5 – Chunking and the Chunk-Size Sweep

A whole Wikipedia page is too big to embed as a single vector if we want precision, but truncating to a fixed chunk size also throws information away. So instead of picking one number, this study sweeps six chunk sizes: 512, 1024, 2048, 4000, 8000 characters and full-page, all of them with a 64-character overlap. The splitter walks through the page text in steps of `chunk_size` characters, breaking at the nearest space so that chunks do not cut words. Each chunk inherits the parent page’s title, links and cleaned HTML region. The chunk’s plain text is what FAISS indexes.

Sweeping the chunk size matters because the answer to a multi-hop question can be hidden in two opposite ways: either by chunks that are too small (the gold sentence sits in chunk $i + 1$ but FAISS only scored chunk i) or by chunks that are too large (the gold sentence is buried in thirty paragraphs of unrelated text). The full-page setting is included as the upper bound on coverage.

F. Phase 6 – LLM Generation

The LLM stage uses Qwen/Qwen3.5-9B [17] as the reader, loaded through the HuggingFace Transformers library [14]. Qwen-3.5-9B is an instruction-tuned model, so it sits in the same family as instruction-tuned LLMs such as FLAN-T5 [16]. The prompt is the standard “Answer the question using the context only” template. Generation EM uses a normalisation (lowercase, strip punctuation, drop articles), and Generation F1 is the standard SQuAD-style token-level F1 of the generated answer against the HotpotQA gold answer.

G. Phase 7 – Evaluation Setup: What We Measured and Why

Different metrics answer different questions, so it is important to pick the right one for the right job. The five metrics used in this study split into two natural groups: three retrieval-side metrics that score the context the system pulled in, and two generation-side metrics that score the final string the LLM wrote. Each one is defined first, and then a short note explains what it is good for and what it is not.

- **Retrieval EM.** A binary indicator that returns 1 if the gold answer string appears anywhere inside the retrieved context (after stripping HTML), and 0 otherwise. It tells us whether the system at least made the answer visible to the LLM. It is biased upward when the context window grows, because longer contexts are more likely to contain any given short string by chance.
- **Retrieval F1.** Token-level F1 between the gold answer and the entire context, computed via Eq. (3). It is reported here for completeness because earlier RAG papers used it, but as Section IV explains, this metric is dominated by context length and is not very informative in the chunk-size sweep.
- **Supporting Recall (SR).** The fraction of the two gold supporting pages that the retriever actually returned. SR is the cleanest signal for multi-hop retrieval, because it directly measures whether the system pulled in the bridge pages instead of just any page that mentions the answer.
- **Generation EM.** Exact match between the LLM’s final generated answer and the gold answer, after a normalisation. Of all the metrics used here, this is the one closest to “did the system actually answer the user’s question correctly?”.
- **Generation F1.** Token-level F1 of the generated answer against the gold answer. It is more forgiving than EM, so it gives partial credit when the model gets “Delhi, India” instead of just “Delhi”.

$$F_1 = \frac{2PR}{P+R}, \quad P = \frac{|t_{\text{pred}} \cap t_{\text{gold}}|}{|t_{\text{pred}}|}, \quad R = \frac{|t_{\text{pred}} \cap t_{\text{gold}}|}{|t_{\text{gold}}|}, \quad (3)$$

where $t_{(\cdot)}$ is the multiset of lowercased whitespace-split tokens.

Every metric is computed on the same 7,926 bridge questions for every chunk size and every retrieval system. So the three systems are scored on identical questions with identical seed values. The only thing that varies between rows is the system and the chunk size.

IV. RESULT AND DISCUSSION

A. Headline Numbers

The full chunk-size $\times$ system grid for all four core metrics is collected in Table I. There are six rows per metric block (one per chunk size) and three columns (one per system).

TABLE I
CHUNK-SIZE SWEEP ON 7,926 HOTPOTQA BRIDGE QUESTIONS OVER A
24,473-PAGE WIKIPEDIA CORPUS.
QWEN-3.5-9B READER, $K = 5$, EXPAND_K = 5.

Chunk	Simple RAG	HtmlRAG	HyperRAG
Retrieval EM			
512	0.347	0.166	0.462
1K	0.398	0.230	0.511
2K	0.438	0.311	0.554
4K	0.479	0.389	0.609
8K	0.512	0.471	0.658
Full page	0.596	0.647	0.885
Supporting Recall			
512	0.602	0.602	0.781
1K	0.590	0.590	0.754
2K	0.590	0.590	0.732
4K	0.590	0.590	0.724
8K	0.589	0.589	0.721
Full page	0.614	0.614	0.799
Generation EM (Qwen-3.5-9B)			
512	0.299	0.218	0.316
1K	0.318	0.244	0.333
2K	0.335	0.277	0.356
4K	0.351	0.315	0.383
8K	0.359	0.338	0.389
Full page	0.360	0.326	0.371
Generation F1 (Qwen-3.5-9B)			
512	0.393	0.292	0.413
1K	0.418	0.325	0.436
2K	0.437	0.360	0.459
4K	0.452	0.400	0.485
8K	0.459	0.429	0.491
Full page	0.459	0.410	0.470

B. Retrieval EM and Supporting Recall

HyperRAG leads on Retrieval EM at every chunk size, and it leads on Supporting Recall at every chunk size as well. The Supporting Recall lead is the cleanest piece of evidence for the main HyperRAG claim. Across the six chunk settings HyperRAG sits in the 0.721 to 0.799 band, while Simple RAG and HtmlRAG (which share the same FAISS retrieval) sit in 0.589 to 0.614. That is a steady gain of about 13 to 19 percentage points. The pattern is plotted in Fig. 4, and it is the kind of gap one would expect from a method that pulls in bridge pages an embedding model alone would miss.

The Retrieval EM picture, plotted in Fig. 5, has a different shape. EM rises with chunk size for every system. At 512 characters HyperRAG reaches 0.462, at 8K it reaches 0.658,

and at full-page it reaches 0.885. So as the chunks get longer, more of the gold answer strings end up sitting somewhere inside the retrieved context. This sounds dramatic, and partly it is, but it is also a property of how EM is computed in this setup. When the context is one or several entire pages, almost any short gold answer is going to be a substring somewhere. The number that does *not* chase chunk size is Supporting Recall, which is why it is the better discriminator for multi-hop retrieval.

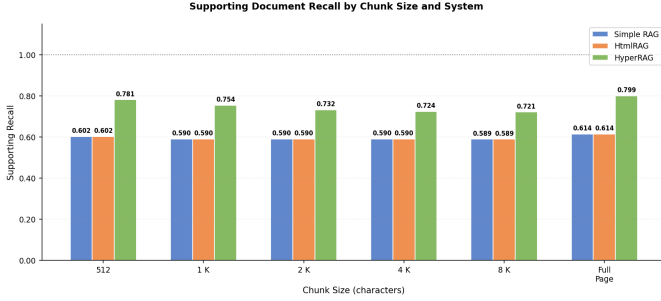


Fig. 4. Supporting Recall by chunk size.

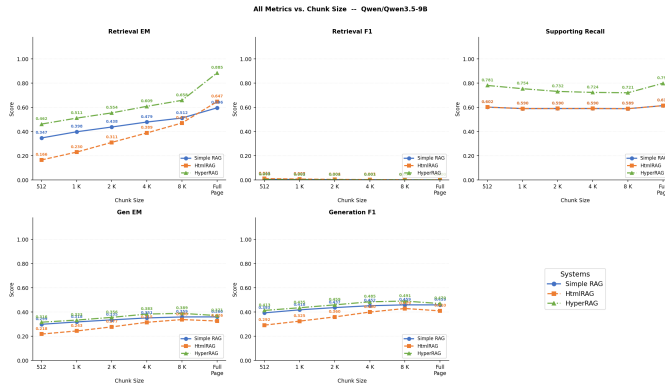


Fig. 5. All five metrics plotted across the six chunk sizes. Retrieval EM, Generation EM and Generation F1 rise with chunk size, Retrieval F1 falls because the context grows much faster than the answer, and Supporting Recall is largely flat once the system is fixed.

C. Generation Quality with Qwen-3.5-9B

The main finding for the generation layer is that HyperRAG wins on both generation metrics at every chunk size. Generation EM at the operationally useful 8K-character setting is 0.389 for HyperRAG, 0.359 for Simple RAG and 0.338 for HtmlRAG. Generation F1 at the same setting is 0.491, 0.459 and 0.429. The margins look small, but they stay consistent. Across all six chunk sizes the HyperRAG-vs-Simple-RAG gap on Generation EM stays in a range of about 1.1 to 3.2 percentage points, and Fig. 6 plots the curves directly.

Three things matter here. First, generation quality saturates above about 4K to 8K characters. The jump from 4K to 8K is small, and the jump from 8K to full-page is essentially zero or slightly negative. The full-page setting actually *drops* HyperRAG’s Generation EM from 0.389 at 8K down to 0.371,

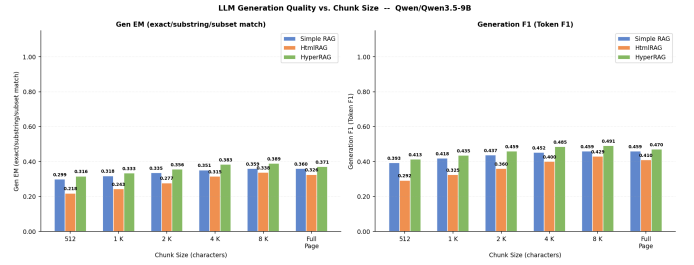


Fig. 6. Generation EM and Generation F1 across the six chunk sizes. HyperRAG sits above Simple RAG at every chunk size, and HtmlRAG sits below both of them.

which suggests that beyond a certain context length the LLM is paying more for distractors than it gains from coverage. Second, HtmlRAG underperforms throughout. The cleaned-HTML representation introduces extra whitespace, tag markers and formatting noise, and short answers (names, dates, places) are sensitive to that kind of noise. Third, Simple RAG and HyperRAG track each other closely, but HyperRAG sits a few points above. The graph expansion pulls in chunks that the LLM can sometimes use even when retrieval EM was already high.

D. The Extraction Gap

A clear gap separates retrieval EM from generation EM at every configuration. At full-page granularity HyperRAG retrieves the gold answer string for 88.5% of the questions (Retrieval EM = 0.885) but generates the right answer for only 37.1% of them (Generation EM = 0.371). That is a 51-percentage-point gap. Fig. 7 plots the gap directly. The gap shrinks slightly with bigger chunks, but it never closes, and it is consistent across systems. This says less about retrieval and more about the reader: even when the gold answer is sitting somewhere in the context, a 9B-parameter LLM still fails to extract it about half of the time. Closing this gap is a reader-side problem (better prompting, an answer-span re-ranker, or just a stronger LLM), not a retrieval-side problem.

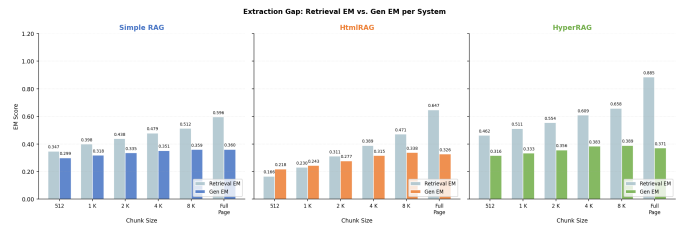


Fig. 7. Extraction gap. The bars show how much retrieval EM survives after the LLM step. The gap shrinks with larger chunks but never fully closes.

E. Side-by-Side Retrieval vs Generation

Fig. 8 shows retrieval EM and generation EM next to each other at the different chunk setting. HyperRAG sits at the top of both bars: highest Retrieval EM (0.885) and highest Generation EM (0.371). Simple RAG follows at 0.596 / 0.36.

HtmlRAG trails at 0.647 / 0.326. The figure makes the same point as the table but at a glance: the ordering of the systems is the same in both phases of the pipeline.

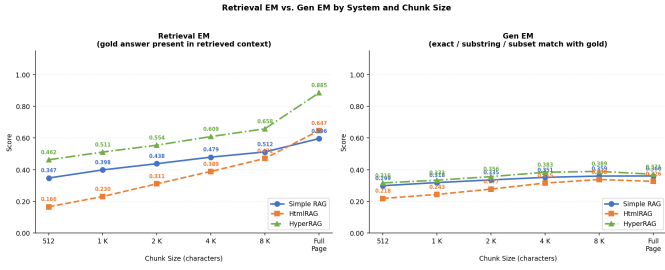


Fig. 8. Retrieval EM versus Generation EM. HyperRAG leads on both phases.

F. Retrieval F1 is Not Informative

Across the table, Retrieval F1 is below 0.015 in every cell, and it actually falls as chunk size grows. This is not a retrieval failure. It is the metric punishing length. F1 compares one to three gold tokens against tens of thousands of retrieved tokens, so the precision term is essentially zero. The headline retrieval and generation conclusions in this paper rely on Supporting Recall, Retrieval EM, Generation EM and Generation F1. Retrieval F1 is reported in the table only because it appears in earlier RAG literature.

G. What the Sweep Says Together

The grid in Table I supports four conclusions. (1) HyperRAG’s graph expansion produces a steady 12 to 17 percentage-point lead on Supporting Recall regardless of chunk size, which is the cleanest evidence that the hyperlink signal is doing real work. (2) HyperRAG also wins on Generation EM and Generation F1 at every chunk size, by a smaller but consistent margin. (3) Generation quality saturates above 4K to 8K characters, so the operationally interesting setting for production-style work is around 4K, and the full-page setting is mostly an academic upper bound. (4) HtmlRAG, despite the structural-markup motivation, consistently does worst on generation in this setup. The additional tags hurt extraction more than they help.

V. CONCLUSION

This paper showed that one of the cheapest possible additions to a standard RAG pipeline, walking one hop along the hyperlink graph that already lives inside HTML, measurably improves multi-hop retrieval and end-to-end answer quality on HotpotQA. Across 7,926 bridge questions and a six-point chunk-size sweep with Qwen-3.5-9B as the reader, HyperRAG held a steady 12 to 17 percentage-point lead on Supporting Recall, the highest Retrieval EM at every chunk size (peaking at 0.885 at full-page), the highest Generation EM at every chunk size (peaking at 0.389 at 8K characters), and the highest Generation F1 at every chunk size (peaking at 0.491 at 8K characters). None of this needed extra training, LLM passes to build a knowledge graph, or any new annotation. The edges

came from `<a href>` tags that were sitting inside the HTML the whole time.

The paper also showed two structural findings that go beyond the headline numbers. First, generation quality saturates above 4K to 8K characters of context. Bigger chunks bring diminishing returns, and the full-page setting actually slightly reduces HyperRAG’s Generation EM. Second, a clear extraction gap of about 33 percentage points separates retrieval EM from generation EM, even when the gold answer is guaranteed to be in the context. That gap is reader-side, not retriever-side, and it is the single most useful lever for further improvement.

Three things are worth trying next. The first is a 2-hop version of the graph expansion, to check if pages that are two links away from the top-K also help, or if they just add noise. The second is a small answer-span re-ranker after retrieval, so that the LLM is pointed at the exact sentence that holds the answer instead of having to find it on its own. This should help close the extraction gap. The third is to try a bigger reader, such as Llama-3-70B or DeepSeek-V3, to see how much of the gap is really about model size.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. NeurIPS*, 2020.
- [2] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, “HotpotQA: A dataset for diverse, explainable multi-hop question answering,” in *Proc. EMNLP*, 2018, pp. 2369–2380.
- [3] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. EMNLP*, 2020, pp. 6769–6781.
- [4] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [5] G. Izacard and E. Grave, “Leveraging passage retrieval with generative models for open-domain question answering,” in *Proc. EACL*, 2021, pp. 874–880.
- [6] Y. Tan, J. Ma, B. Wang, R. Xu, and Z. Tan, “HtmlRAG: HTML is better than plain text for modeling retrieved web knowledge in RAG systems,” *arXiv:2411.02959*, 2024.
- [7] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “From local to global: A graph RAG approach to query-focused summarization,” *arXiv:2404.16130*, 2024.
- [8] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, “HippoRAG: Neurobiologically inspired long-term memory for large language models,” in *Proc. NeurIPS*, 2024.
- [9] H. Liu et al., “HopRAG: Multi-hop reasoning for LLM-based retrieval-augmented generation,” in *Proc. ACL*, 2025.
- [10] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, “RAPTOR: Recursive abstractive processing for tree-organized retrieval,” in *Proc. ICLR*, 2024.
- [11] A. Asai, K. Hashimoto, H. Hajishirzi, R. Socher, and C. Xiong, “Learning to retrieve reasoning paths over Wikipedia graph for question answering,” in *Proc. ICLR*, 2020.
- [12] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [13] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX,” in *Proc. SciPy*, 2008, pp. 11–15.
- [14] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” in *Proc. EMNLP (Demos)*, 2020, pp. 38–45.
- [15] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions,” in *Proc. ACL*, 2023, pp. 10014–10037.

- [16] H. W. Chung et al., “Scaling instruction-finetuned language models,” *arXiv:2210.11416*, 2022.
- [17] Qwen Team, “Qwen technical report,” *arXiv:2309.16609*, 2023.
- [18] R. Prajapati, “HyperRAG project repository,” GitHub repository. [Online]. Available: <https://github.com/roviso/HyperRag.git>